Programação de Sistemas Linux

Aula 11 — Comunicação entre Processos (IPC)

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br

INSTITUTO FEDERAL
Santa Catarina
Câmpus Florianópolis

Notes

Abertura

Notes

Onde estamos

Aula 10 (feito)

- *Thread vs.* processo
- Condição de corrida
- *Mutex*, semáforos, variáveis de condição
- *Deadlock*

Aula 11 (hoje)

- *Pipes* anônimos e FIFOs
- Memória compartilhada POSIX
- Filas de mensagens POSIX
- Como escolher o mecanismo certo

Estrutura da aula

Hoje (2h): teoria e demonstrações.
Próximo encontro (2h): **Laboratório 10**, no laboratório.



Notes

Objetivos da aula

Ao final desta aula, você será capaz de:

- ✓ Explicar por que processos (ao contrário de threads) precisam de um mecanismo explícito para trocar dados
- ✓ Criar e usar um **pipe** anônimo entre processos aparentados, fechando corretamente as pontas não usadas
- ✓ Criar e usar uma **FIFO** para comunicar processos *sem* parentesco
- ✓ Compartilhar memória entre processos com `shm_open`/`mmap`, sincronizando com um **semáforo nomeado**
- ✓ Enviar e receber mensagens com **prioridade** por uma fila de mensagens POSIX
- ✓ Escolher o mecanismo de IPC adequado a um cenário, com base em custo e característica dos dados



Notes

Pipes anônimos

Notes

Pipe: um canal unidirecional entre parentes

Definição

Um **pipe** é um canal de comunicação **unidirecional**, gerenciado pelo kernel, com um buffer limitado. `pipe()` devolve **dois** descritores de arquivo: `fd[0]` (leitura) e `fd[1]` (escrita).

- Só existe entre processos que compartilham os descritores — na prática, **pai e filho após um** `fork()`
- Escrever no pipe cheio **bloqueia** o escritor; ler do pipe vazio **bloqueia** o leitor — não é preciso espera ocupada
- Quando **todas** as pontas de escrita são fechadas, o leitor vê **EOF** (`read()` retorna 0)



Notes

Um pipe básico: pai lê, filho escreve

```c
int fd[2];
pipe(fd);                       /* fd[0] = leitura, fd[1] = escrita */

pid_t pid = fork();
if (pid == 0) {                 /* FILHO: so escreve */
    close(fd[0]);
    write(fd[1], msg, strlen(msg) + 1);
    close(fd[1]);
} else {                        /* PAI: so le */
    close(fd[1]);
    read(fd[0], buf, sizeof(buf));
    close(fd[0]);
}

```

```
$ ./pipe_demo
[PAI] recebi do pipe (22 bytes): Ola do filho, PID=305

```



Notes

A regra de ouro: feche as pontas que você não usa

Por que fechar `fd[0]`/`fd[1]` importa

O kernel só sinaliza **EOF** a um leitor quando **nenhum** processo no sistema ainda tem a ponta de escrita aberta — inclusive se for o próprio **leitor**, por esquecimento, que mantém uma cópia dela aberta sem usar.

- Cada processo deve fechar, logo após o `fork()`, a ponta do pipe que **não** vai usar
- Esquecer isso não dá erro de compilação nem de execução — **o programa simplesmente trava**, esperando um EOF que nunca chega



Notes

Vendo o travamento ao vivo

```c
/* BUG: o pai esqueceu de fazer close(fd[1]) */
while ((n = read(fd[0], buf, sizeof(buf) - 1)) > 0) {
    printf("recebi: %s", buf);
}
/* nunca chega aqui -- read() fica bloqueado para sempre */

```

```
$ timeout 3 ./pipe_loop_bug
[PAI] lendo em loop até EOF (vai travar)...
$ echo $?
124    <- o timeout teve que matar: o read() nunca viu EOF

```

- O filho escreveu tudo e fechou **sua** ponta de escrita — mas o **pai** ainda tinha `fd[1]` aberto, então o kernel nunca considerou o pipe “sem escritores”
- Corrigido: basta o pai chamar `close(fd[1])` antes do laço de leitura — o mesmo programa, com essa única linha, termina normalmente e imprime as 3 linhas

Notes

FIFOs (named pipes)

Notes

FIFO: um pipe com nome no sistema de arquivos

Definição

Uma **FIFO** (*named pipe*) tem a mesma semântica de um pipe anônimo, mas existe como uma entrada no sistema de arquivos (`mkfifo()`), com um nome. Isso permite que processos **sem parentesco** se encontrem para se comunicar.

- `ls -l` mostra o tipo `p` (*pipe*) — não é um arquivo normal: **abrir uma FIFO para leitura bloqueia** até alguém abri-la para escrita, e vice-versa
- O **conteúdo** nunca é gravado em disco — só os dados em trânsito pelo buffer do kernel, exatamente como no pipe anônimo
- `mkfifo /caminho/fifo` (shell) ou `mkfifo()` (C)



Notes

FIFO entre dois processos independentes

```c
/* reader.c */                      /* writer.c */
fd = open("/tmp/psl_fifo",          fd = open("/tmp/psl_fifo",
          O_RDONLY);  /* bloqueia */          O_WRONLY); /* bloqueia */
while ((n = read(fd, buf, N)) > 0)  write(fd, msg, len);
    printf("recebi: %s", buf);      close(fd);

```

```
$ mkfifo /tmp/psl_fifo
$ ./fifo_reader &          # em outro terminal/processo
[READER] abrindo para leitura (vai bloquear até alguem abrir p/ escrita)
    ...
$ ./fifo_writer
[WRITER] conectado! enviando 3 mensagens...
[READER] conectado! lendo até EOF...
[READER] recebi: mensagem 1 do writer (PID=351)
[READER] recebi: mensagem 2 do writer (PID=351)
[READER] recebi: mensagem 3 do writer (PID=351)
[READER] EOF -- writer fechou o FIFO. Encerrando.

```



Notes

Pipe anônimo *vs.* FIFO

	Pipe anônimo	**FIFO**
Quem pode usar	só parentes (via `fork`)	**qualquer processo**
Como se encontram	herdam o *fd* do pai	abrem o **mesmo caminho**
Existe no sistema de arquivos?	não	sim (`ls -l` mostra tipo `p`)
Dados ficam em disco?	não	não — só em trânsito

- A FIFO resolve exatamente a limitação do pipe anônimo: dois programas que **não** compartilham ancestral podem se combinar, desde que concordem com antecedência num **caminho**



Notes

Memória compartilhada POSIX

Notes

Memória compartilhada: sem copiar nada

Definição

Memória compartilhada mapeia a **mesma** região física de memória no espaço de endereçamento de **vários** processos. Depois do mapeamento, ler/escrever é só um acesso a ponteiro — **sem** chamada de sistema, sem copiar nada pelo kernel.

- É o mecanismo de IPC **mais rápido** que existe — e o único que não envolve nenhuma cópia de dados em uso normal
- Mas não sincroniza nada por conta própria: ler antes do outro processo escrever dá o mesmo tipo de **condição de corrida** vista na Aula 10 — só que agora entre processos



Notes

Criando e mapeando: três passos

```c
int fd = shm_open("/psl_shm_demo", O_CREAT | O_RDWR, 0666);
ftruncate(fd, N_INTS * sizeof(int));      /* shm nasce com tamanho 0! */
int *vetor = mmap(NULL, tamanho, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
close(fd);   /* o mapeamento continua valido depois de fechar o fd */

```

- `shm_open`: cria/abre um objeto com **nome** (aparece em `/dev/shm/`), análogo ao `mkfifo` da FIFO
- `ftruncate`: define o **tamanho** — obrigatório, o objeto nasce vazio
- `mmap`: mapeia o objeto no espaço de endereçamento — `vetor[i]` passa a ser **memória de verdade**



Notes

Sincronizando entre processos: semáforo *nomeado*

Por que não `sem_init()`, como na Aula 10?

`sem_init()` cria um semáforo que vive numa variável do **seu** processo — inútil para dois processos com espaços de endereçamento separados. Para IPC, o semáforo também precisa de um **nome** no sistema: `sem_open()`.

```
1 /* produtor: cria, comeca "zerado" */
2 sem_t *sem = sem_open("/psl_sem_demo", O_CREAT, 0666, 0);
3 ...
4 sem_post(sem);      /* "pode ler, os dados estao prontos" */
5
6 /* consumidor: so abre (sem O_CREAT) */
7 sem_t *sem = sem_open("/psl_sem_demo", 0);
8 sem_wait(sem);      /* bloqueia aqui ate o produtor avisar */
9
```



Notes

Produtor e consumidor, com e sem sincronização

```
1 $ ./shm_produtor &  (escreve 10 inteiros, so depois faz sem_post)
2 $ ./shm_consumidor   (sem_wait ... so le depois do aviso)
3 [CONSUMIDOR] esperando o produtor avisar (sem_wait)...
4 [CONSUMIDOR] acordei! lendo a memoria compartilhada:
5   ...
6 [CONSUMIDOR] soma total = 285        <- correto
7
```

```
1 $ ./shm_criar_vazio      # cria o shm, mas NAO escreve nada ainda
2 $ ./shm_consumidor_bug   # le sem esperar sem_wait nenhum
3 [CONSUMIDOR-BUG] lendo SEM esperar o produtor...
4 [CONSUMIDOR-BUG] soma total = 0 (esperado: 285)    <- errado, em
      silencio
5
```

- Sem sincronização, o programa **não trava nem erra** — só calcula o valor errado, exatamente como a condição de corrida de threads da Aula 10



Notes

O preço de copiar: pipe *vs.* memória compartilhada

Transferindo 20 MB entre dois processos, de formas equivalentes:

```
1 $ ./bench_pipe     # 20 MB em pedacos de 64 KB, via write()/read()
2 [PIPE] transferidos 20971520 bytes em 10.88 ms
3
4 $ ./bench_shm      # 20 MB ja compartilhados, so percorridos
5 [SHM]  20971520 bytes acessiveis diretamente, percorridos em 1.06 ms
6
```

- O pipe faz **duas** cópias por `write()`/`read()` (processo → kernel, kernel → processo); a memória compartilhada faz **zero**
- Por isso é o mecanismo preferido para **grandes volumes** de dados, ou quando o mesmo dado precisa ser acessado repetidas vezes



Notes

Filas de mensagens POSIX

Notes

Fila de mensagens: pacotes discretos, com prioridade

Definição

Uma **fila de mensagens** POSIX transporta **mensagens inteiras** (não um fluxo de bytes, como o pipe) e entrega sempre a de **maior prioridade** primeiro — mesmo que tenha chegado depois das outras.

```
1 mqd_t mq = mq_open("/psl_mq_demo", O_CREAT | O_RDWR, 0666, &attr);
2 mq_send(mq, "mensagem", tamanho, prioridade);
3 ssize_t n = mq_receive(mq, buf, sizeof(buf), &prioridade_recebida);
4
```


Notes

Prioridade na prática: a ordem de saída muda

```
1 [ENVIO] enviei 'tarefa normal' com prioridade 1
2 [ENVIO] enviei 'tarefa de fundo' com prioridade 0
3 [ENVIO] enviei 'ALERTA URGENTE' com prioridade 9
4
5 [RECEBIMENTO] ordem de ENVIO foi: normal(1), fundo(0), urgente(9)
6 [RECEBIMENTO] lendo a fila agora, em ordem de PRIORIDADE:
7   prioridade=9 -> "ALERTA URGENTE (prioridade 9)"
8   prioridade=1 -> "tarefa normal (prioridade 1)"
9   prioridade=0 -> "tarefa de fundo (prioridade 0)"
10
```

- A mensagem de prioridade 9 foi a **última** a ser enviada e a **primeira** a ser recebida
- Entre mensagens de mesma prioridade, a ordem é FIFO (a de chegada) — a prioridade só desempata quando é diferente
- Útil para filas de tarefas com urgências distintas (ex.: um *alerta* que deve furar a frente de tarefas de rotina)

Notes

Escolhendo o mecanismo certo

Notes

Qual mecanismo usar?

	Pipe	FIFO	Mem. compartilhada	Fila de mensagens
Entre processos	parentes	quaisquer	quaisquer	quaisquer
Unidade de dados	fluxo de bytes	fluxo de bytes	região de memória	mensagens discretas
Sincronização	embutida (bloqueia)	embutida (bloqueia)	**manual** (semáforo)	embutida (bloqueia)
Prioridade?	não	não	não	**sim**
Custo por byte	2 cópias	2 cópias	**0 cópias**	2 cópias

- Fluxo simples entre pai e filho → **pipe**. Processos sem parentesco → **FIFO**. Volume grande ou acesso repetido → **memória compartilhada**. Tarefas discretas com urgência → **fila de mensagens**

Notes

Encerramento

Notes

Síntese da aula

Mecanismo	Pergunta que responde
Pipe anônimo	Como meu filho me manda dados, sem nenhum nome no sistema?
FIFO	Como dois processos sem parentesco se encontram para trocar dados?
Memória compartilhada	Como evito copiar um volume grande de dados repetidamente?
Semáforo nomeado	Como sincronizo dois processos que não compartilham memória de threads?
Fila de mensagens	Como garanto que o alerta urgente não espera na fila atrás do trivial?

A lição que atravessa a aula

Threads sincronizam porque **compartilham** memória por padrão; processos têm que **escolher** um mecanismo para compartilhar *qualquer* coisa — e essa escolha tem consequências de desempenho e de semântica bem diferentes.

Notes

Atividade prática — próximo encontro

Laboratório 10 — Comunicação entre Processos (2h)

Parte A, presencial:

1. Pipe anônimo: comunicação pai–filho e o bug do `close()` esquecido
2. FIFO entre dois processos independentes
3. Memória compartilhada com semáforo nomeado — produtor/consumidor

Parte B, extraclasse: pipe bidirecional (requisição/resposta), fila de mensagens com prioridade, e um comparativo de desempenho pipe *vs.* memória compartilhada.

Atenção

No exercício do pipe, teste **de propósito** o bug do `close()` esquecido antes de corrigi-lo — é mais fácil reconhecer esse travamento depois de já ter visto ele acontecer.



Notes

Próxima aula

Aula 12 — Sinais e Gerenciamento de Tempo

- Sinais: envio, tratamento (`signal`/`sigaction`), máscaras
- Sinais entre processos como forma de IPC (notificação, sem dados)
- Temporizadores e medição de tempo (`alarm`, `timerfd`, `clock_gettime`)

Bibliografia de apoio

- Manuais: `man 7 pipe`, `man 7 fifo`, `man 7 shm_overview`, `man 7 mq_overview`, `man 3 sem_open`
- The Linux Programming Interface (Michael Kerrisk) — capítulos 44, 48–52
- STEVENS, W. R.; RAGO, S. A. *Advanced Programming in the UNIX Environment* — capítulo 15



Notes

That’s all folks!

INSTITUTO FEDERAL
Santa Catarina
Câmpus Florianópolis

Notes

Notes